```
set<int> s;
for( int i = 0; i < 1000000; ++i )
    s.insert( s.end( ), i );
```

There are several versions of `erase`:

```
int erase( const Object & x );
iterator erase( iterator itr );
iterator erase( iterator start, iterator end );
```

The first one-parameter `erase` removes `x` (if found) and returns the number of items actually removed, which is obviously either 0 or 1. The second one-parameter `erase` behaves the same as in `vector` and `list`. It removes the object at the position given by the `iterator`, returns an `iterator` representing the element that followed `itr` immediately prior to the call to `erase`, and invalidates `itr`, which becomes stale. The two-parameter `erase` behaves the same as in a `vector` or `list`, removing all the items starting at `start`, up to but not including the item at `end`.

For searching, rather than a `contains` routine that returns a Boolean variable, the `set` provides a `find` routine that returns an `iterator` representing the location of the item (or the endmarker if the search fails). This provides considerably more information, at no cost in running time. The signature of `find` is

```
iterator find( const Object & x ) const;
```

By default, ordering uses the `less<Object>` function object, which itself is implemented by invoking `operator<` for the `Object`. An alternative ordering can be specified by instantiating the `set` template with a function object type. For instance, we can create a `set` that stores `string` objects, ignoring case distinctions by using the `CaseInsensitiveCompare` function object coded in Figure 1.25. In the following code, the `set s` has size 1:

```
set<string,CaseInsensitiveCompare> s;
s.insert( "Hello" ); s.insert( "HeLLo" );
cout << "The size is: " << s.size( ) << endl;
```

4.8.2 Maps

A `map` is used to store a collection of ordered entries that consists of keys and their values. Keys must be unique, but several keys can map to the same values. Thus values need not be unique. The keys in the `map` are maintained in logically sorted order.

The `map` behaves like a `set` instantiated with a `pair`, whose comparison function refers only to the key.[5] Thus it supports `begin`, `end`, `size`, and `empty`, but the underlying iterator is a key-value pair. In other words, for an `iterator itr`, `*itr` is of type `pair<KeyType,ValueType>`. The `map` also supports `insert`, `find`, and `erase`. For `insert`, one must provide a `pair<KeyType,ValueType>` object. Although `find` requires only a key, the

[5] Like a `set`, an optional template parameter can be used to specify a comparison function that differs from `less<KeyType>`.

iterator it returns references a pair. Using only these operations is often not worthwhile because the syntactic baggage can be expensive.

Fortunately, the map has an important extra operation that yields simple syntax. The array-indexing operator is overloaded for maps as follows:

```
ValueType & operator[] ( const KeyType & key );
```

The semantics of operator[] are as follows. If key is present in the map, a reference to the corresponding value is returned. If key is not present in the map, it is inserted with a default value into the map and then a reference to the inserted default value is returned. The default value is obtained by applying a zero-parameter constructor or is zero for the primitive types. These semantics do not allow an accessor version of operator[], so operator[] cannot be used on a map that is constant. For instance, if a map is passed by constant reference, inside the routine, operator[] is unusable.

The code snippet in Figure 4.68 illustrates two techniques to access items in a map. First, observe that at line 3, the left-hand side invokes operator[], thus inserting "Pat" and a double of value 0 into the map, and returning a reference to that double. Then the assignment changes that double inside the map to 75000. Line 4 outputs 75000. Unfortunately, line 5 inserts "Jan" and a salary of 0.0 into the map and then prints it. This may or may not be the proper thing to do, depending on the application. If it is important to distinguish between items that are in the map and those not in the map, or if it is important not to insert into the map (because it is immutable), then an alternate approach shown at lines 7 to 12 can be used. There we see a call to find. If the key is not found, the iterator is the endmarker and can be tested. If the key is found, we can access the second item in the pair referenced by the iterator, which is the value associated with the key. We could also assign to itr->second if, instead of a const_iterator, itr is an iterator.

4.8.3 Implementation of set and map

C++ requires that set and map support the basic insert, erase, and find operations in logarithmic worst-case time. Consequently, the underlying implementation is a balanced

```
1     map<string,double> salaries;
2
3     salaries[ "Pat" ] = 75000.00;
4     cout << salaries[ "Pat" ] << endl;
5     cout << salaries[ "Jan" ] << endl;
6
7     map<string,double>::const_iterator itr;
8     itr = salaries.find( "Chris" );
9     if( itr == salaries.end( ) )
10        cout << "Not an employee of this company!" << endl;
11    else
12        cout << itr->second << endl;
```

Figure 4.68 Accessing values in a map

binary search tree. Typically, an AVL tree is not used; instead, top-down red-black trees, which are discussed in Section 12.2, are often used.

An important issue in implementing `set` and `map` is providing support for the iterator classes. Of course, internally, the iterator maintains a pointer to the "current" node in the iteration. The hard part is efficiently advancing to the next node. There are several possible solutions, some of which are listed here:

1. When the iterator is constructed, have each iterator store as its data an array containing the `set` items. This doesn't work: It makes it impossible to efficiently implement any of the routines that return an iterator after modifying the `set`, such as some of the versions of `erase` and `insert`.
2. Have the iterator maintain a stack storing nodes on the path to the current node. With this information, one can deduce the next node in the iteration, which is either the node in the current node's right subtree that contains the minimum item or the nearest ancestor that contains the current node in its left subtree. This makes the iterator somewhat large and makes the iterator code clumsy.
3. Have each node in the search tree store its parent in addition to the children. The iterator is not as large, but there is now extra memory required in each node, and the code to iterate is still clumsy.
4. Have each node maintain extra links: one to the next smaller, and one to the next larger node. This takes space, but the iteration is very simple to do, and it is easy to maintain these links.
5. Maintain the extra links only for nodes that have `nullptr` left or right links by using extra Boolean variables to allow the routines to tell if a left link is being used as a standard binary search tree left link or a link to the next smaller node, and similarly for the right link (Exercise 4.49). This idea is called a **threaded tree** and is used in many of the STL implementations.

4.8.4 An Example That Uses Several Maps

Many words are similar to other words. For instance, by changing the first letter, the word `wine` can become `dine`, `fine`, `line`, `mine`, `nine`, `pine`, or `vine`. By changing the third letter, `wine` can become `wide`, `wife`, `wipe`, or `wire`, among others. By changing the fourth letter, `wine` can become `wind`, `wing`, `wink`, or `wins`, among others. This gives 15 different words that can be obtained by changing only one letter in `wine`. In fact, there are over 20 different words, some more obscure. We would like to write a program to find all words that can be changed into at least 15 other words by a single one-character substitution. We assume that we have a dictionary consisting of approximately 89,000 different words of varying lengths. Most words are between 6 and 11 characters. The distribution includes 8,205 six-letter words, 11,989 seven-letter words, 13,672 eight-letter words, 13,014 nine-letter words, 11,297 ten-letter words, and 8,617 eleven-letter words. (In reality, the most changeable words are three-, four-, and five-letter words, but the longer words are the time-consuming ones to check.)

The most straightforward strategy is to use a `map` in which the keys are words and the values are vectors containing the words that can be changed from the key with a

```
void printHighChangeables( const map<string,vector<string>> & adjacentWords,
                           int minWords = 15 )
{
    for( auto & entry : adjacentWords )
    {
        const vector<string> & words = entry.second;

        if( words.size( ) >= minWords )
        {
            cout << entry.first << " (" << words.size( ) << "):";
            for( auto & str : words )
                cout << " " << str;
            cout << endl;
        }
    }
}
```

Figure 4.69 Given a `map` containing words as keys and a `vector` of words that differ in only one character as values, output words that have `minWords` or more words obtainable by a one-character substitution

one-character substitution. The routine in Figure 4.69 shows how the `map` that is eventually produced (we have yet to write code for that part) can be used to print the required answers. The code uses a range `for` loop to step through the `map` and views entries that are pairs consisting of a word and a vector of words. The constant references at lines 4 and 6 are used to replace complicated expressions and avoid making unneeded copies.

The main issue is how to construct the `map` from an array that contains the 89,000 words. The routine in Figure 4.70 is a straightforward function to test if two words are identical except for a one-character substitution. We can use the routine to provide the simplest algorithm for the `map` construction, which is a brute-force test of all pairs of words. This algorithm is shown in Figure 4.71.

If we find a pair of words that differ in only one character, we can update the `map` at lines 12 and 13. The idiom we are using at line 12 is that `adjWords[str]` represents the `vector` of words that are identical to `str`, except for one character. If we have previously seen `str`, then it is in the `map`, and we need only add the new word to the `vector` in the `map`, and we do this by calling `push_back`. If we have never seen `str` before, then the act of using `operator[]` places it in the `map`, with a `vector` of size 0, and returns this `vector`, so the `push_back` updates the `vector` to be size 1. All in all, a super-slick idiom for maintaining a `map` in which the value is a collection.

The problem with this algorithm is that it is slow and takes 97 seconds on our computer. An obvious improvement is to avoid comparing words of different lengths. We can do this by grouping words by their length, and then running the previous algorithm on each of the separate groups.

To do this, we can use a second `map`! Here the key is an integer representing a word length, and the value is a collection of all the words of that length. We can use a `vector` to

```
// Returns true if word1 and word2 are the same length
// and differ in only one character.
bool oneCharOff( const string & word1, const string & word2 )
{
    if( word1.length( ) != word2.length( ) )
        return false;

    int diffs = 0;

    for( int i = 0; i < word1.length( ); ++i )
        if( word1[ i ] != word2[ i ] )
            if( ++diffs > 1 )
                return false;

    return diffs == 1;
}
```

Figure 4.70 Routine to check if two words differ in only one character

```
// Computes a map in which the keys are words and values are vectors of words
// that differ in only one character from the corresponding key.
// Uses a quadratic algorithm.
map<string,vector<string>> computeAdjacentWords( const vector<string> & words )
{
    map<string,vector<string>> adjWords;

    for( int i = 0; i < words.size( ); ++i )
        for( int j = i + 1; j < words.size( ); ++j )
            if( oneCharOff( words[ i ], words[ j ] ) )
            {
                adjWords[ words[ i ] ].push_back( words[ j ] );
                adjWords[ words[ j ] ].push_back( words[ i ] );
            }

    return adjWords;
}
```

Figure 4.71 Function to compute a `map` containing words as keys and a `vector` of words that differ in only one character as values. This version runs in 1.5 minutes on an 89,000-word dictionary.

store each collection, and the same idiom applies. The code is shown in Figure 4.72. Line 8 shows the declaration for the second `map`, lines 11 and 12 populate the `map`, and then an extra loop is used to iterate over each group of words. Compared to the first algorithm, the second algorithm is only marginally more difficult to code and runs in 18 seconds, or about six times as fast.

```
// Computes a map in which the keys are words and values are vectors of words
// that differ in only one character from the corresponding key.
// Uses a quadratic algorithm, but speeds things up a little by
// maintaining an additional map that groups words by their length.
map<string,vector<string>> computeAdjacentWords( const vector<string> & words )
{
    map<string,vector<string>> adjWords;
    map<int,vector<string>> wordsByLength;

      // Group the words by their length
    for( auto & thisWord : words )
        wordsByLength[ thisWord.length( ) ].push_back( thisWord );

      // Work on each group separately
    for( auto & entry : wordsByLength )
    {
        const vector<string> & groupsWords = entry.second;

        for( int i = 0; i < groupsWords.size( ); ++i )
            for( int j = i + 1; j < groupsWords.size( ); ++j )
                if( oneCharOff( groupsWords[ i ], groupsWords[ j ] ) )
                {
                    adjWords[ groupsWords[ i ] ].push_back( groupsWords[ j ] );
                    adjWords[ groupsWords[ j ] ].push_back( groupsWords[ i ] );
                }
    }

    return adjWords;
}
```

Figure 4.72 Function to compute a `map` containing words as keys and a `vector` of words that differ in only one character as values. It splits words into groups by word length. This version runs in 18 seconds on an 89,000-word dictionary.

Our third algorithm is more complex and uses additional `maps`! As before, we group the words by word length, and then work on each group separately. To see how this algorithm works, suppose we are working on words of length 4. First we want to find word pairs, such as `wine` and `nine`, that are identical except for the first letter. One way to do this is as follows: For each word of length 4, remove the first character, leaving a three-character word representative. Form a `map` in which the key is the representative, and the value is a `vector` of all words that have that representative. For instance, in considering the first character of the four-letter word group, representative `"ine"` corresponds to `"dine"`, `"fine"`, `"wine"`, `"nine"`, `"mine"`, `"vine"`, `"pine"`, `"line"`. Representative `"oot"` corresponds to `"boot"`, `"foot"`, `"hoot"`, `"loot"`, `"soot"`, `"zoot"`. Each individual `vector` that is a value in this latest `map` forms a clique of words in which any word can be changed to any other word by a one-character substitution, so after this latest `map` is constructed, it is easy to traverse it and add entries to the original `map` that is being computed. We would then proceed to

the second character of the four-letter word group, with a new `map`, and then the third character, and finally the fourth character.

The general outline is

```
for each group g, containing words of length len
    for each position p (ranging from 0 to len-1)
    {
        Make an empty map<string,vector<string>> repsToWords
        for each word w
        {
            Obtain w's representative by removing position p
            Update repsToWords
        }
        Use cliques in repsToWords to update adjWords map
    }
```

Figure 4.73 contains an implementation of this algorithm. The running time improves to two seconds. It is interesting to note that although the use of the additional `maps` makes the algorithm faster, and the syntax is relatively clean, the code makes no use of the fact that the keys of the `map` are maintained in sorted order.

```
// Computes a map in which the keys are words and values are vectors of words
// that differ in only one character from the corresponding key.
// Uses an efficient algorithm that is O(N log N) with a map
map<string,vector<string>> computeAdjacentWords( const vector<string> & words )
{
    map<string,vector<string>> adjWords;
    map<int,vector<string>> wordsByLength;

      // Group the words by their length
    for( auto & str : words )
        wordsByLength[ str.length( ) ].push_back( str );

      // Work on each group separately
    for( auto & entry : wordsByLength )
    {
        const vector<string> & groupsWords = entry.second;
        int groupNum = entry.first;

        // Work on each position in each group
        for( int i = 0; i < groupNum; ++i )
```

Figure 4.73 Function to compute a map containing words as keys and a vector of words that differ in only one character as values. This version runs in 2 seconds on an 89,000-word dictionary.

```
            {
                // Remove one character in specified position, computing representative.
                // Words with same representatives are adjacent; so populate a map ...
                map<string,vector<string>> repToWord;

                for( auto & str : groupsWords )
                {
                    string rep = str;
                    rep.erase( i, 1 );
                    repToWord[ rep ].push_back( str );
                }

                // and then look for map values with more than one string
                for( auto & entry : repToWord )
                {
                    const vector<string> & clique = entry.second;
                    if( clique.size( ) >= 2 )
                        for( int p = 0; p < clique.size( ); ++p )
                            for( int q = p + 1; q < clique.size( ); ++q )
                            {
                                adjWords[ clique[ p ] ].push_back( clique[ q ] );
                                adjWords[ clique[ q ] ].push_back( clique[ p ] );
                            }
                }
            }
        }
        return adjWords;
    }
```

Figure 4.73 *(continued)*

As such, it is possible that a data structure that supports the `map` operations but does not guarantee sorted order can perform better, since it is being asked to do less. Chapter 5 explores this possibility and discusses the ideas behind the alternative map implementation that C++11 adds to the Standard Library, known as an `unordered_map`. An unordered map reduces the running time of the implementation from 2 sec to 1.5 sec.

Summary

We have seen uses of trees in operating systems, compiler design, and searching. Expression trees are a small example of a more general structure known as a **parse tree**, which is a central data structure in compiler design. Parse trees are not binary, but are relatively simple extensions of expression trees (although the algorithms to build them are not quite so simple).

Search trees are of great importance in algorithm design. They support almost all the useful operations, and the logarithmic average cost is very small. Nonrecursive implementations of search trees are somewhat faster, but the recursive versions are sleeker, more elegant, and easier to understand and debug. The problem with search trees is that their performance depends heavily on the input being random. If this is not the case, the running time increases significantly, to the point where search trees become expensive linked lists.

We saw several ways to deal with this problem. AVL trees work by insisting that all nodes' left and right subtrees differ in heights by at most one. This ensures that the tree cannot get too deep. The operations that do not change the tree, as insertion does, can all use the standard binary search tree code. Operations that change the tree must restore the tree. This can be somewhat complicated, especially in the case of deletion. We showed how to restore the tree after insertions in $O(\log N)$ time.

We also examined the splay tree. Nodes in splay trees can get arbitrarily deep, but after every access the tree is adjusted in a somewhat mysterious manner. The net effect is that any sequence of M operations takes $O(M \log N)$ time, which is the same as a balanced tree would take.

B-trees are balanced M-way (as opposed to 2-way or binary) trees, which are well suited for disks; a special case is the 2–3 tree ($M = 3$), which is another way to implement balanced search trees.

In practice, the running time of all the balanced-tree schemes, while slightly faster for searching, is worse (by a constant factor) for insertions and deletions than the simple binary search tree, but this is generally acceptable in view of the protection being given against easily obtained worst-case input. Chapter 12 discusses some additional search tree data structures and provides detailed implementations.

A final note: By inserting elements into a search tree and then performing an inorder traversal, we obtain the elements in sorted order. This gives an $O(N \log N)$ algorithm to sort, which is a worst-case bound if any sophisticated search tree is used. We shall see better ways in Chapter 7, but none that have a lower time bound.

Exercises

4.1 For the tree in Figure 4.74:
a. Which node is the root?
b. Which nodes are leaves?

4.2 For each node in the tree of Figure 4.74:
a. Name the parent node.
b. List the children.
c. List the siblings.
d. Compute the depth.
e. Compute the height.

4.3 What is the depth of the tree in Figure 4.74?

4.4 Show that in a binary tree of N nodes, there are $N + 1$ `nullptr` links representing children.

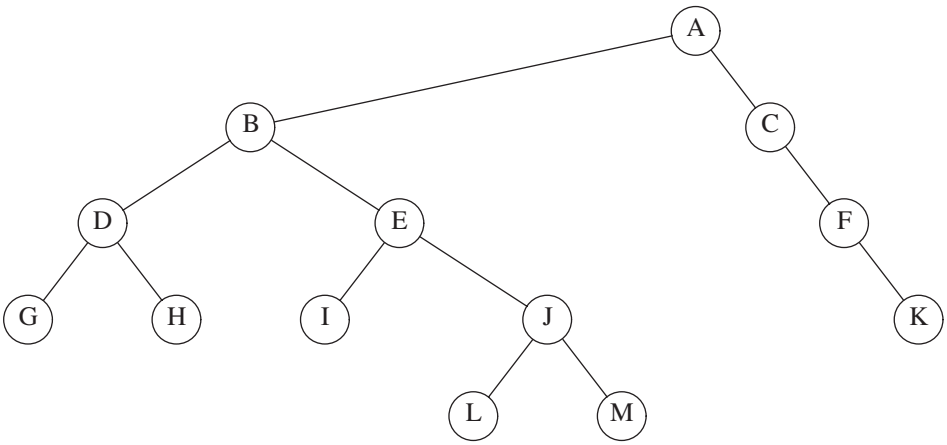

Figure 4.74 Tree for Exercises 4.1 to 4.3

4.5 Show that the maximum number of nodes in a binary tree of height h is $2^{h+1} - 1$.

4.6 A *full node* is a node with two children. Prove that the number of full nodes plus one is equal to the number of leaves in a nonempty binary tree.

4.7 Suppose a binary tree has leaves $l_1, l_2, \ldots, l_M$ at depths $d_1, d_2, \ldots, d_M$, respectively. Prove that $\sum_{i=1}^{M} 2^{-d_i} \leq 1$ and determine when the equality is true.

4.8 Give the prefix, infix, and postfix expressions corresponding to the tree in Figure 4.75.

4.9 a. Show the result of inserting 3, 1, 4, 6, 9, 2, 5, 7 into an initially empty binary search tree.
b. Show the result of deleting the root.

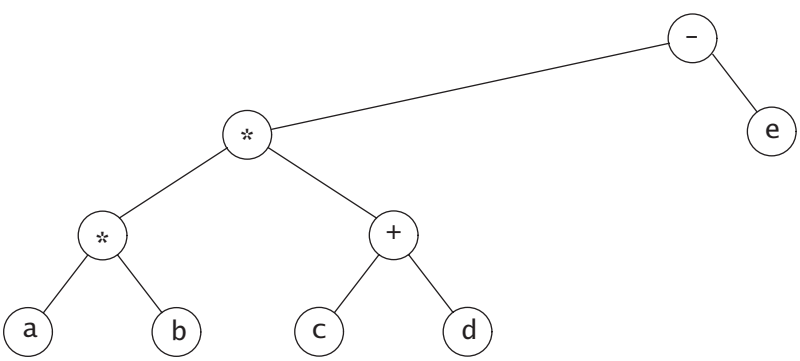

Figure 4.75 Tree for Exercise 4.8

4.10 Let $f(N)$ be the average number of full nodes in an N-node binary search tree.
a. Determine the values of $f(0)$ and $f(1)$.
b. Show that for $N > 1$

$$f(N) = \frac{N-2}{N} + \frac{1}{N}\sum_{i=0}^{N-1}(f(i) + f(N-i-1))$$

c. Show (by induction) that $f(N) = (N-2)/3$ is a solution to the equation in part (b), with the initial conditions in part (a).
d. Use the results of Exercise 4.6 to determine the average number of leaves in an N-node binary search tree.

4.11 Write an implementation of the `set` class, with associated iterators using a binary search tree. Add to each node a link to the parent node.

4.12 Write an implementation of the `map` class by storing a data member of type `set<Pair<KeyType,ValueType>>`.

4.13 Write an implementation of the `set` class, with associated iterators using a binary search tree. Add to each node a link to the next smallest and next largest node. To make your code simpler, add a header and tail node which are not part of the binary search tree, but help make the linked list part of the code simpler.

4.14 Suppose you want to perform an experiment to verify the problems that can be caused by random `insert/remove` pairs. Here is a strategy that is not perfectly random, but close enough. You build a tree with N elements by inserting N elements chosen at random from the range 1 to $M = \alpha N$. You then perform N^2 pairs of insertions followed by deletions. Assume the existence of a routine, `randomInteger(a,b)`, which returns a uniform random integer between `a` and `b` inclusive.
a. Explain how to generate a random integer between 1 and M that is not already in the tree (so a random insertion can be performed). In terms of N and α, what is the running time of this operation?
b. Explain how to generate a random integer between 1 and M that is already in the tree (so a random deletion can be performed). What is the running time of this operation?
c. What is a good choice of α? Why?

4.15 Write a program to evaluate empirically the following strategies for removing nodes with two children:
a. Replace with the largest node, X, in T_L and recursively remove X.
b. Alternately replace with the largest node in T_L and the smallest node in T_R, and recursively remove the appropriate node.
c. Replace with either the largest node in T_L or the smallest node in T_R (recursively removing the appropriate node), making the choice randomly.

Which strategy seems to give the most balance? Which takes the least CPU time to process the entire sequence?

4.16 Redo the binary search tree class to implement lazy deletion. Note carefully that this affects all of the routines. Especially challenging are `findMin` and `findMax`, which must now be done recursively.

** **4.17** Prove that the depth of a random binary search tree (depth of the deepest node) is $O(\log N)$, on average.

4.18 * a. Give a precise expression for the minimum number of nodes in an AVL tree of height h.
b. What is the minimum number of nodes in an AVL tree of height 15?

4.19 Show the result of inserting 2, 1, 4, 5, 9, 3, 6, 7 into an initially empty AVL tree.

* **4.20** Keys $1, 2, \ldots, 2^k - 1$ are inserted in order into an initially empty AVL tree. Prove that the resulting tree is perfectly balanced.

4.21 Write the remaining procedures to implement AVL single and double rotations.

4.22 Design a linear-time algorithm that verifies that the height information in an AVL tree is correctly maintained and that the balance property is in order.

4.23 Write a nonrecursive function to insert into an AVL tree.

4.24 Show that the deletion algorithm in Figure 4.47 is correct

4.25 a. How many bits are required per node to store the height of a node in an N-node AVL tree?
b. What is the smallest AVL tree that overflows an 8-bit height counter?

4.26 Write the functions to perform the double rotation without the inefficiency of doing two single rotations.

4.27 Show the result of accessing the keys 3, 9, 1, 5 in order in the splay tree in Figure 4.76.

4.28 Show the result of deleting the element with key 6 in the resulting splay tree for the previous exercise.

4.29 a. Show that if all nodes in a splay tree are accessed in sequential order, the resulting tree consists of a chain of left children.

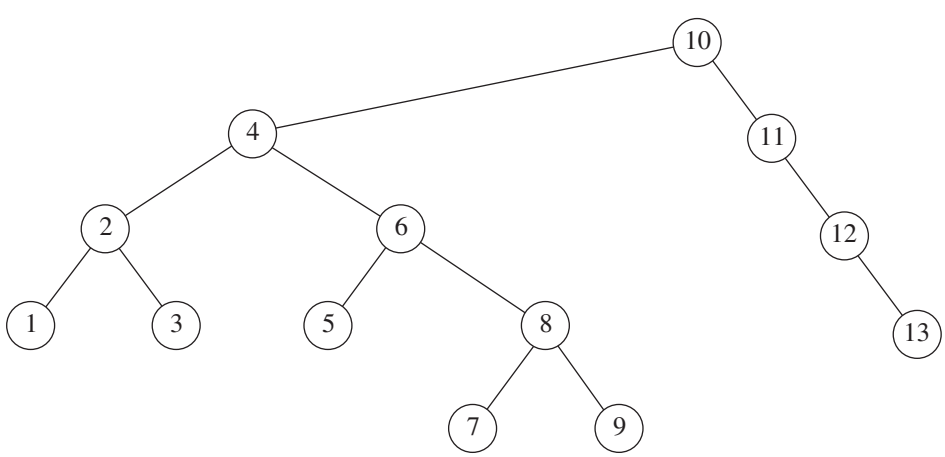

Figure 4.76 Tree for Exercise 4.27

** b. Show that if all nodes in a splay tree are accessed in sequential order, then the total access time is $O(N)$, regardless of the initial tree.

4.30 Write a program to perform random operations on splay trees. Count the total number of rotations performed over the sequence. How does the running time compare to AVL trees and unbalanced binary search trees?

4.31 Write efficient functions that take only a pointer to the root of a binary tree, T, and compute

a. the number of nodes in T
b. the number of leaves in T
c. the number of full nodes in T

What is the running time of your routines?

4.32 Design a recursive linear-time algorithm that tests whether a binary tree satisfies the search tree order property at every node.

4.33 Write a recursive function that takes a pointer to the root node of a tree T and returns a pointer to the root node of the tree that results from removing all leaves from T.

4.34 Write a function to generate an N-node random binary search tree with distinct keys 1 through N. What is the running time of your routine?

4.35 Write a function to generate the AVL tree of height h with fewest nodes. What is the running time of your function?

4.36 Write a function to generate a perfectly balanced binary search tree of height h with keys 1 through $2^{h+1} - 1$. What is the running time of your function?

4.37 Write a function that takes as input a binary search tree, T, and two keys, k_1 and k_2, which are ordered so that $k_1 \leq k_2$, and prints all elements X in the tree such that $k_1 \leq Key(X) \leq k_2$. Do not assume any information about the type of keys except that they can be ordered (consistently). Your program should run in $O(K + \log N)$ average time, where K is the number of keys printed. Bound the running time of your algorithm.

4.38 The larger binary trees in this chapter were generated automatically by a program. This was done by assigning an (x, y) coordinate to each tree node, drawing a circle around each coordinate (this is hard to see in some pictures), and connecting each node to its parent. Assume you have a binary search tree stored in memory (perhaps generated by one of the routines above) and that each node has two extra fields to store the coordinates.

a. The x coordinate can be computed by assigning the inorder traversal number. Write a routine to do this for each node in the tree.
b. The y coordinate can be computed by using the negative of the depth of the node. Write a routine to do this for each node in the tree.
c. In terms of some imaginary unit, what will the dimensions of the picture be? How can you adjust the units so that the tree is always roughly two-thirds as high as it is wide?
d. Prove that using this system no lines cross, and that for any node, X, all elements in X's left subtree appear to the left of X and all elements in X's right subtree appear to the right of X.

4.39 Write a general-purpose tree-drawing program that will convert a tree into the following graph-assembler instructions:
a. *Circle*(X, Y)
b. *DrawLine*(i, j)
The first instruction draws a circle at (X, Y), and the second instruction connects the ith circle to the jth circle (circles are numbered in the order drawn). You should either make this a program and define some sort of input language or make this a function that can be called from any program. What is the running time of your routine?

4.40 Write a routine to list out the nodes of a binary tree in *level-order*. List the root, then nodes at depth 1, followed by nodes at depth 2, and so on. You must do this in linear time. Prove your time bound.

4.41 ★ a. Write a routine to perform insertion into a B-tree.
★ b. Write a routine to perform deletion from a B-tree. When an item is deleted, is it necessary to update information in the internal nodes?
★ c. Modify your insertion routine so that if an attempt is made to add into a node that already has M entries, a search is performed for a sibling with less than M children before the node is split.

4.42 A B^**-tree* of order M is a B-tree in which each interior node has between $2M/3$ and M children. Describe a method to perform insertion into a B^*-tree.

4.43 Show how the tree in Figure 4.77 is represented using a child/sibling link implementation.

4.44 Write a procedure to traverse a tree stored with child/sibling links.

4.45 Two binary trees are similar if they are both empty or both nonempty and have similar left and right subtrees. Write a function to decide whether two binary trees are similar. What is the running time of your function?

4.46 Two trees, T_1 and T_2, are *isomorphic* if T_1 can be transformed into T_2 by swapping left and right children of (some of the) nodes in T_1. For instance, the two trees in Figure 4.78 are isomorphic because they are the same if the children of A, B, and G, but not the other nodes, are swapped.
a. Give a polynomial time algorithm to decide if two trees are isomorphic.

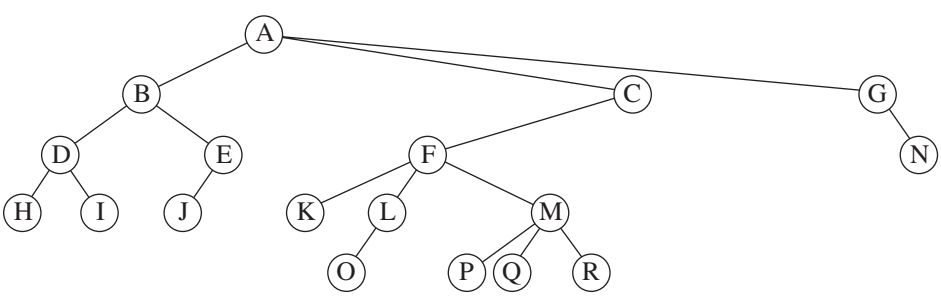

Figure 4.77 Tree for Exercise 4.43

Figure 4.78 Two isomorphic trees

* b. What is the running time of your program (there is a linear solution)?

4.47 * a. Show that via AVL single rotations, any binary search tree T_1 can be transformed into another search tree T_2 (with the same items).

* b. Give an algorithm to perform this transformation using $O(N \log N)$ rotations on average.

** c. Show that this transformation can be done with $O(N)$ rotations, worst-case.

4.48 Suppose we want to add the operation `findKth` to our repertoire. The operation `findKth(k)` returns the `k`th smallest item in the tree. Assume all items are distinct. Explain how to modify the binary search tree to support this operation in $O(\log N)$ average time, without sacrificing the time bounds of any other operation.

4.49 Since a binary search tree with N nodes has $N + 1$ `nullptr` pointers, half the space allocated in a binary search tree for pointer information is wasted. Suppose that if a node has a `nullptr` left child, we make its left child link to its inorder predecessor, and if a node has a `nullptr` right child, we make its right child link to its inorder successor. This is known as a *threaded tree* and the extra links are called *threads*.

a. How can we distinguish threads from real children pointers?

b. Write routines to perform insertion and deletion into a tree threaded in the manner described above.

c. What is the advantage of using threaded trees?

4.50 Write a program that reads a C++ source code file and outputs a list of all identifiers (that is, variable names, but not keywords, that are not found in comments or string constants) in alphabetical order. Each identifier should be output with a list of line numbers on which it occurs.

4.51 Generate an index for a book. The input file consists of a set of index entries. Each line consists of the string `IX:`, followed by an index entry name enclosed in braces, followed by a page number that is enclosed in braces. Each `!` in an index entry name represents a sublevel. A `|(` represents the start of a range, and a `|)` represents the end of the range. Occasionally, this range will be the same page. In that case, output only a single page number. Otherwise, do not collapse or expand ranges on your own. As an example, Figure 4.79 shows sample input and Figure 4.80 shows the corresponding output.

```
IX: {Series|(}            {2}
IX: {Series!geometric|(}  {4}
IX: {Euler's constant}    {4}
IX: {Series!geometric|)}  {4}
IX: {Series!arithmetic|(} {4}
IX: {Series!arithmetic|)} {5}
IX: {Series!harmonic|(}   {5}
IX: {Euler's constant}    {5}
IX: {Series!harmonic|)}   {5}
IX: {Series|)}            {5}
```

Figure 4.79 Sample input for Exercise 4.51

```
Euler's constant: 4, 5
Series: 2-5
   arithmetic: 4-5
   geometric: 4
   harmonic: 5
```

Figure 4.80 Sample output for Exercise 4.51

References

More information on binary search trees, and in particular the mathematical properties of trees, can be found in the two books by Knuth, [22] and [23].

Several papers deal with the lack of balance caused by biased deletion algorithms in binary search trees. Hibbard's paper [19] proposed the original deletion algorithm and established that one deletion preserves the randomness of the trees. A complete analysis has been performed only for trees with three nodes [20] and four nodes [5]. Eppinger's paper [14] provided early empirical evidence of nonrandomness, and the papers by Culberson and Munro [10], [11] provided some analytical evidence (but not a complete proof for the general case of intermixed insertions and deletions).

Adelson-Velskii and Landis [1] proposed AVL trees. Recently it was shown that for AVL trees, if rebalancing is performed only on insertions, and not on deletions, under certain circumstances the resulting structure still maintains a depth of $O(\log M)$ where M is the number of insertions [28]. Simulation results for AVL trees, and variants in which the height imbalance is allowed to be at most k for various values of k, are presented in [21]. Analysis of the average search cost in AVL trees is incomplete, but some results are contained in [24].

[3] and [8] considered self-adjusting trees like the type in Section 4.5.1. Splay trees are described in [29].

B-trees first appeared in [6]. The implementation described in the original paper allows data to be stored in internal nodes as well as leaves. The data structure we have described

is sometimes known as a B^+-tree. A survey of the different types of B-trees is presented in [9]. Empirical results of the various schemes are reported in [17]. Analysis of 2–3 trees and B-trees can be found in [4], [13], and [32].

Exercise 4.17 is deceptively difficult. A solution can be found in [15]. Exercise 4.29 is from [32]. Information on B*-trees, described in Exercise 4.42, can be found in [12]. Exercise 4.46 is from [2]. A solution to Exercise 4.47 using $2N - 6$ rotations is given in [30]. Using threads, à la Exercise 4.49, was first proposed in [27]. k-d trees, which handle multidimensional data, were first proposed in [7] and are discussed in Chapter 12.

Other popular balanced search trees are red-black trees [18] and weight-balanced trees [26]. More balanced-tree schemes can be found in the books [16] and [25].

1. G. M. Adelson-Velskii and E. M. Landis, "An Algorithm for the Organization of Information," *Soviet. Mat. Doklady,* 3 (1962), 1259–1263.
2. A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *The Design and Analysis of Computer Algorithms,* Addison-Wesley, Reading, Mass., 1974.
3. B. Allen and J. I. Munro, "Self Organizing Search Trees," *Journal of the ACM,* 25 (1978), 526–535.
4. R. A. Baeza-Yates, "Expected Behaviour of B^+-trees under Random Insertions," *Acta Informatica,* 26 (1989), 439–471.
5. R. A. Baeza-Yates, "A Trivial Algorithm Whose Analysis Isn't: A Continuation," *BIT,* 29 (1989), 88–113.
6. R. Bayer and E. M. McCreight, "Organization and Maintenance of Large Ordered Indices," *Acta Informatica,* 1 (1972), 173–189.
7. J. L. Bentley, "Multidimensional Binary Search Trees Used for Associative Searching," *Communications of the ACM,* 18 (1975), 509–517.
8. J. R. Bitner, "Heuristics that Dynamically Organize Data Structures," *SIAM Journal on Computing,* 8 (1979), 82–110.
9. D. Comer, "The Ubiquitous B-tree," *Computing Surveys,* 11 (1979), 121–137.
10. J. Culberson and J. I. Munro, "Explaining the Behavior of Binary Search Trees under Prolonged Updates: A Model and Simulations," *Computer Journal,* 32 (1989), 68–75.
11. J. Culberson and J. I. Munro, "Analysis of the Standard Deletion Algorithms in Exact Fit Domain Binary Search Trees," *Algorithmica,* 5 (1990), 295–311.
12. K. Culik, T. Ottman, and D. Wood, "Dense Multiway Trees," *ACM Transactions on Database Systems,* 6 (1981), 486–512.
13. B. Eisenbath, N. Ziviana, G. H. Gonnet, K. Melhorn, and D. Wood, "The Theory of Fringe Analysis and Its Application to 2–3 Trees and B-trees," *Information and Control,* 55 (1982), 125–174.
14. J. L. Eppinger, "An Empirical Study of Insertion and Deletion in Binary Search Trees," *Communications of the ACM,* 26 (1983), 663–669.
15. P. Flajolet and A. Odlyzko, "The Average Height of Binary Trees and Other Simple Trees," *Journal of Computer and System Sciences,* 25 (1982), 171–213.
16. G. H. Gonnet and R. Baeza-Yates, *Handbook of Algorithms and Data Structures,* 2d ed., Addison-Wesley, Reading, Mass., 1991.
17. E. Gudes and S. Tsur, "Experiments with B-tree Reorganization," *Proceedings of ACM SIGMOD Symposium on Management of Data* (1980), 200–206.